Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Guía Práctica — Sesión 8

CRUD de tablas dependientes: Productos y su categoría

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	8 · 1 de octubre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Angular 22 · TypeScript · Spring Boot 4	Proyecto base	PharmaSoft (sesión 7) + PharmaBackend
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. Situación de la sesión

La semana pasada el jefe de almacén empezó a usar PharmaSoft para mantener las categorías de la botica. Ahora necesita registrar los **productos**, y cada producto pertenece a una categoría. En la hoja de cálculo que usaban antes, la categoría se escribía a mano: había «Analgesicos», «analgésicos» y «ANALG.», y nadie sabía cuántos productos tenía cada una. Además, alguien borró una categoría que todavía tenía productos y esos productos quedaron sin clasificar.

Hoy la SPA debe impedir esos problemas. El usuario no escribirá la categoría: la **elegirá de una lista** con las categorías que realmente existen y están activas. Si intenta algo que rompe la relación entre las tablas, la aplicación se lo explicará con el mensaje del backend.

¿Cómo garantizas que cada producto quede asociado a una categoría válida, tanto al registrarlo como al modificarlo o darlo de baja?

Reto 01 (actividad práctica, en clase): implementar el CRUD de Productos conectado a PharmaBackend y resolver la selección de su categoría con una lista desplegable validada.

Reto 02 (actividad autónoma): probar altas, cambios y bajas válidas e inválidas según las dependencias entre Categorías y Productos, y reportar los hallazgos.

1.1 Resultado de aprendizaje y criterio que se trabaja

Elemento	Descripción
Resultado de aprendizaje de la unidad	Construye una SPA empresarial conectada al backend REST, con layout y navegación, CRUD de tablas independientes y dependientes, formularios cabecera–detalle, consultas y reportes; integra seguridad mediante JWT en el backend y gestión de sesión, guards e interceptores en el frontend.
Criterio 2 del producto	Implementa CRUD de tablas dependientes mediante selección de datos relacionados, listas desplegables y validación de dependencias.

1.2 Punto de partida

Requisito	Cómo verificarlo
SPA PharmaSoft de la sesión 7 con el módulo Categorías funcionando y fusionado en `develop`.	`git log --oneline develop` muestra tus commits de la sesión 7.
PharmaBackend actualizado desde `master` (incluye CORS, paginación y Flyway).	En Swagger, `GET /api/v1/productos?pagina=0&tamanio=5` devuelve un objeto con `contenido`.
Al menos tres categorías, una de ellas inactiva, y algunos productos registrados.	Regístralos desde la SPA (categorías) y desde Swagger (productos).

2. Actividad práctica

Volvamos al almacén: el jefe de almacén quiere registrar productos sin volver a escribir categorías a mano ni perder productos cuando se modifica el catálogo. Con el Reto 01 agregarás a PharmaSoft el módulo Productos, cuya particularidad es depender de otra tabla. Trabajarás paso a paso y con el acompañamiento del docente.

Campo	Contenido
Título	Módulo frontend: CRUD de Productos con selección de categoría y validación de dependencias
Objetivo	Implementa el CRUD de Productos en la SPA PharmaSoft relacionando cada producto con su categoría mediante una lista desplegable validada, y explica las respuestas del backend ante dependencias válidas e inválidas.
Duración	100 minutos de trabajo + 20 minutos de pruebas y retroalimentación

Paso 1. Actualizar los repositorios

Actualiza PharmaBackend y arráncalo. Luego crea tu rama de trabajo en el frontend a partir de `develop`:

```
# Backend
cd PharmaBackend
git checkout master
git pull

# Frontend
cd ../pharma-frontend
git checkout develop
git pull
git checkout -b feature/productos-<apellido>
```

Paso 2. Revisar el contrato de Productos

Antes de escribir código, prueba en Swagger los cinco endpoints. Productos se comporta distinto que Categorías en tres aspectos: **depende** de una categoría, su listado llega **paginado** y la eliminación es una **baja lógica**.

Operación	Petición	Respuestas
Listar	`GET /api/v1/productos?pagina=0&tamanio=10&ordenarPor=nombre&direccion=asc`	200 con `PaginaResponseDTO`. `ordenarPor` admite id, nombre, precio o stock; otro valor → 409.
Consultar	`GET /api/v1/productos/{id}`	200 · 404
Registrar	`POST /api/v1/productos`	201 · 400 · 404 (categoría inexistente) · 409 (nombre repetido)
Actualizar	`PUT /api/v1/productos/{id}`	200 · 400 · 404 · 409
Dar de baja	`DELETE /api/v1/productos/{id}`	204: el producto queda con `estado: false`. Si ya estaba inactivo → 409.

Reglas del `ProductoRequestDTO` que replicarás en el formulario:

Campo	Tipo	Regla
nombre	string	Obligatorio, de 3 a 150 caracteres. Único (409 si se repite).
precio	number	Obligatorio, mayor o igual a 0.01.
stock	number	Obligatorio, entero mayor o igual a 0.
estado	boolean	Obligatorio.
categoriaId	number	Obligatorio y positivo; debe existir (404 si no existe).

Paso 3. Generar el feature Productos

```
ng g c features/productos/pages/producto-list
ng g c features/productos/pages/producto-form
ng g s features/productos/services/producto-service
ng g interface features/productos/models/producto --type=model
```

Al terminar, la carpeta del feature tendrá la misma forma que la de Categorías:

```
src/app/
├── core/models/pagina-response.ts      ← nuevo, reutilizable
└── features/productos/
    ├── models/producto.model.ts
    ├── services/producto-service.ts
    ├── pages/producto-list/    (.ts, .html, .css)
    ├── pages/producto-form/    (.ts, .html, .css)
    └── productos.routes.ts
```

Paso 4. Modelos

El listado paginado se describe con un modelo genérico que servirá también para Clientes y Ventas. Créalo en `core`, porque no pertenece a ningún feature en particular.

Archivo: `src/app/core/models/pagina-response.ts`

```
export interface PaginaResponse<T> {
  contenido: T[];
  pagina: number;
  tamanio: number;
  totalElementos: number;
  totalPaginas: number;
  ultima: boolean;
}
```

El modelo de Producto refleja el `ProductoResponseDTO` y el `ProductoRequestDTO`. Observa que el request solo lleva `categoriaId`, mientras que la respuesta trae además `categoriaNombre` para mostrarlo en la tabla. Los tipos `OrdenProducto` y `Direccion` restringen, desde TypeScript, los valores que acepta el backend.

Archivo: `src/app/features/productos/models/producto.model.ts`

```
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  stock: number;
  estado: boolean;
  categoriaId: number;
  categoriaNombre: string;
  fechaCreacion: string;
  fechaModificacion: string | null;
}

export interface ProductoRequest {
  nombre: string;
  precio: number;
  stock: number;
  estado: boolean;
  categoriaId: number;
}

/** Campos que el backend acepta en ordenarPor. */
export type OrdenProducto = 'id' | 'nombre' | 'precio' | 'stock';
export type Direccion = 'asc' | 'desc';
```

Paso 5. Servicio HTTP

`listar()` envía los parámetros de paginación y orden con `HttpParams`. El método de eliminación se llama `darDeBaja()` para que el nombre refleje lo que realmente hace el backend.

Archivo: `src/app/features/productos/services/producto-service.ts`

```
import { inject, Service } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../../environments/environment';
import { PaginaResponse } from '../../../core/models/pagina-response';
import { Direccion, OrdenProducto, Producto, ProductoRequest } from
'../models/producto.model';

@Service()
export class ProductoService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/productos`;

  listar(pagina: number, tamanio: number, ordenarPor: OrdenProducto,
         direccion: Direccion): Observable<PaginaResponse<Producto>> {
    const params = new HttpParams()
      .set('pagina', pagina)
      .set('tamanio', tamanio)
      .set('ordenarPor', ordenarPor)
      .set('direccion', direccion);
    return this.http.get<PaginaResponse<Producto>>(this.url, { params });
  }

  obtener(id: number): Observable<Producto> {
    return this.http.get<Producto>(`${this.url}/${id}`);
  }

  crear(dto: ProductoRequest): Observable<Producto> {
    return this.http.post<Producto>(this.url, dto);
  }

  actualizar(id: number, dto: ProductoRequest): Observable<Producto> {
    return this.http.put<Producto>(`${this.url}/${id}`, dto);
  }

  /** Baja lógica: el backend marca estado = false. */
  darDeBaja(id: number): Observable<void> {
    return this.http.delete<void>(`${this.url}/${id}`);
  }
}
```

Paso 6. Rutas y menú

Archivo: `src/app/features/productos/productos.routes.ts`

```
import { Routes } from '@angular/router';
import { ProductoList } from './pages/producto-list/producto-list';
import { ProductoForm } from './pages/producto-form/producto-form';

export const PRODUCTOS_ROUTES: Routes = [
  { path: '', component: ProductoList, title: 'Productos' },
  { path: 'nuevo', component: ProductoForm, title: 'Nuevo producto' },
  { path: ':id/editar', component: ProductoForm, title: 'Editar producto' },
];
```

En `app.routes.ts`, agrega la ruta dentro de `children` de MainLayout, después de la de Categorías:

```
{
  path: 'productos',
  loadChildren: () =>
    import('./features/productos/productos.routes').then(m => m.PRODUCTOS_ROUTES),
},
```

Y agrega la opción al menú, en `core/config/menu.ts`:

```
{ etiqueta: 'Productos', ruta: '/productos', icono: '🏷️' },
```

Paso 7. Listado con información relacionada

El listado guarda en signals los parámetros de la consulta (página, tamaño, campo y dirección de orden) y el resultado paginado. Además carga las categorías para el filtro. El `computed` `productos` aplica ese filtro **sobre la página actual**: la API no recibe `categoriaId` en el listado, una limitación que analizarás en la actividad autónoma.

Archivo: `src/app/features/productos/pages/producto-list/producto-list.ts`

```
import { Component, computed, inject, OnInit, signal } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { RouterLink } from '@angular/router';
import { PaginaResponse } from '../../../../core/models/pagina-response';
import { mensajeError } from '../../../../core/utils/http-error';
import { Categoria } from '../../../categorias/models/categoria.model';
import { CategoriaService } from '../../../categorias/services/categoria-service';
import { Direccion, OrdenProducto, Producto } from '../../models/producto.model';
import { ProductoService } from '../../services/producto-service';

@Component({
  selector: 'app-producto-list',
  imports: [RouterLink, CurrencyPipe],
  templateUrl: './producto-list.html',
  styleUrl: './producto-list.css',
})
export class ProductoList implements OnInit {
  private readonly productoService = inject(ProductoService);
  private readonly categoriaService = inject(CategoriaService);

  // Estado de la consulta paginada
  protected readonly pagina = signal(0);
  protected readonly tamanio = signal(10);
  protected readonly ordenarPor = signal<OrdenProducto>('nombre');
  protected readonly direccion = signal<Direccion>('asc');

  protected readonly resultado = signal<PaginaResponse<Producto> | null>(null);
  protected readonly categorias = signal<Categoria[]>([]);
  protected readonly categoriaFiltro = signal<number | null>(null);
  protected readonly cargando = signal(false);
  protected readonly error = signal<string | null>(null);

  /** Filtra por categoría los productos de la página actual. */
  protected readonly productos = computed(() => {
    const filtro = this.categoriaFiltro();
    const lista = this.resultado()?.contenido ?? [];
    return filtro === null ? lista : lista.filter(p => p.categoriaId === filtro);
  });

  ngOnInit(): void {
    this.categoriaService.listar().subscribe({
      next: datos => this.categorias.set(datos),
      error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
    });
    this.cargar();
  }

  cargar(): void {
    this.cargando.set(true);
    this.error.set(null);
    this.productoService
      .listar(this.pagina(), this.tamanio(), this.ordenarPor(), this.direccion())
      .subscribe({
        next: pagina => {
```

```
          this.resultado.set(pagina);
          this.cargando.set(false);
        },
        error: (err: HttpErrorResponse) => {
          this.error.set(mensajeError(err));
          this.cargando.set(false);
        },
      });
  }

  irA(pagina: number): void {
    this.pagina.set(pagina);
    this.cargar();
  }

  cambiarTamanio(valor: string): void {
    this.tamanio.set(Number(valor));
    this.irA(0);
  }

  ordenar(campo: OrdenProducto): void {
    if (this.ordenarPor() === campo) {
      this.direccion.update(d => (d === 'asc' ? 'desc' : 'asc'));
    } else {
      this.ordenarPor.set(campo);
      this.direccion.set('asc');
    }
    this.irA(0);
  }

  filtrarPorCategoria(valor: string): void {
    this.categoriaFiltro.set(valor ? Number(valor) : null);
  }

  darDeBaja(producto: Producto): void {
    if (!confirm(`¿Dar de baja el producto "${producto.nombre}"?`)) {
      return;
    }
    this.productoService.darDeBaja(producto.id).subscribe({
      next: () => this.cargar(),
      error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
    });
  }
}
```

La plantilla muestra la categoría de cada producto con `categoriaNombre`, el precio con `CurrencyPipe` y el paginador con los datos de `PaginaResponse`. El botón «Dar de baja» se deshabilita cuando el producto ya está inactivo.

Archivo: `src/app/features/productos/pages/producto-list/producto-list.html`

```
<section class="page">
  <header class="page-header">
    <h2>Productos</h2>
    <a routerLink="nuevo" class="btn btn-primary">+ Nuevo producto</a>
  </header>

  <div class="filtros">
    <label>
      Categoría
      <select class="input" (change)="filtrarPorCategoria($any($event.target).value)">
        <option value="">Todas</option>
        @for (c of categorias(); track c.id) {
          <option [value]="c.id">{{ c.nombre }}</option>
        }
      </select>
    </label>
    <label>
      Por página
```

```
      <select class="input" [value]="tamanio()"
(change)="cambiarTamanio($any($event.target).value)">
        <option value="5">5</option>
        <option value="10">10</option>
        <option value="20">20</option>
      </select>
    </label>
  </div>

  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  @if (cargando()) {
    <p>Cargando productos…</p>
  } @else {
    <table class="tabla">
      <thead>
        <tr>
          <th><button type="button" class="orden"
(click)="ordenar('nombre')">Nombre</button></th>
          <th>Categoría</th>
          <th><button type="button" class="orden"
(click)="ordenar('precio')">Precio</button></th>
          <th><button type="button" class="orden"
(click)="ordenar('stock')">Stock</button></th>
          <th>Estado</th>
          <th>Acciones</th>
        </tr>
      </thead>
      <tbody>
        @for (p of productos(); track p.id) {
          <tr>
            <td>{{ p.nombre }}</td>
            <td><span class="chip">{{ p.categoriaNombre }}</span></td>
            <td>{{ p.precio | currency: 'PEN' : 'S/ ' }}</td>
            <td>{{ p.stock }}</td>
            <td>
              <span class="badge" [class.inactivo]="!p.estado">{{ p.estado ? 'Activo' :
'Inactivo' }}</span>
            </td>
            <td class="acciones">
              <a [routerLink]="[p.id, 'editar']" class="btn">Editar</a>
              <button type="button" class="btn btn-peligro" [disabled]="!p.estado"
(click)="darDeBaja(p)">
                Dar de baja
              </button>
            </td>
          </tr>
        } @empty {
          <tr><td colspan="6" class="vacio">No hay productos para mostrar.</td></tr>
        }
      </tbody>
    </table>

    @if (resultado(); as r) {
      <footer class="paginador">
        <span>Página {{ r.pagina + 1 }} de {{ r.totalPaginas || 1 }} · {{ r.totalElementos }}
productos</span>
        <button type="button" class="btn" [disabled]="r.pagina === 0" (click)="irA(r.pagina -
1)">Anterior</button>
        <button type="button" class="btn" [disabled]="r.ultima" (click)="irA(r.pagina +
1)">Siguiente</button>
      </footer>
    }
  }
</section>
```

Archivo: `src/app/features/productos/pages/producto-list/producto-list.css`

```
.filtros { display: flex; gap: 16px; margin-bottom: 16px; }
.filtros label { display: flex; flex-direction: column; gap: 4px; font-size: 14px; font-weight:
600; }
.filtros .input { min-width: 180px; }
.orden { background: none; border: 0; color: inherit; font: inherit; font-weight: 700; cursor:
pointer; padding: 0; }
.orden::after { content: ' ↕'; opacity: 0.6; }
.chip { padding: 2px 10px; border-radius: 999px; background: #e9f0f9; color: #0f2e5c; font-size:
13px; }
.acciones { display: flex; gap: 8px; }
.vacio { text-align: center; color: #6b7280; padding: 24px; }
.paginador { display: flex; align-items: center; justify-content: flex-end; gap: 12px; margin-top:
12px; }
```

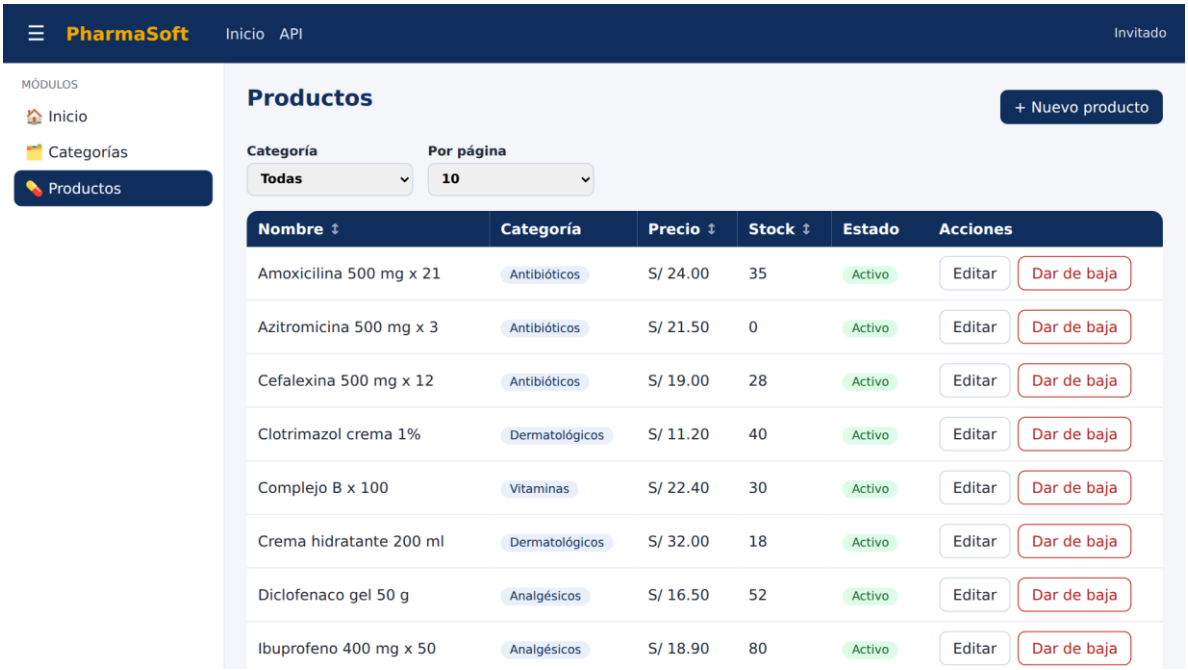


Figura 1. Listado paginado de productos con su categoría. Nota. Elaboración propia.

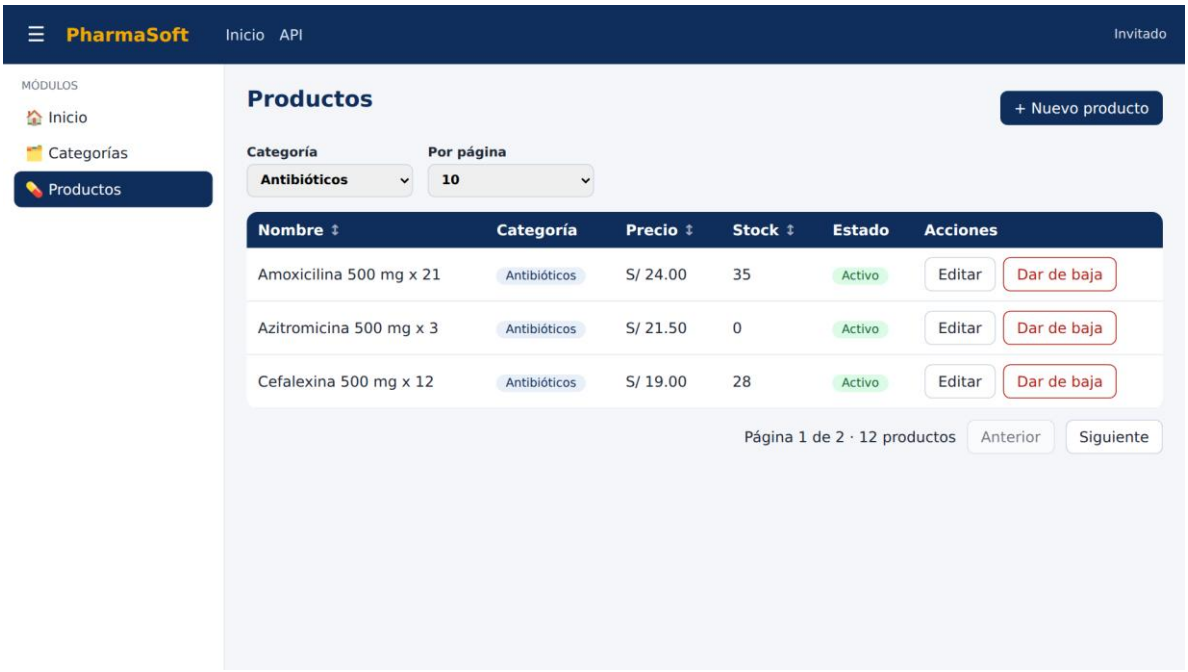


Figura 2. Filtro por categoría aplicado a la página actual. Nota. Elaboración propia.

Paso 8. Formulario con lista desplegable validada

Este es el núcleo de la sesión. Revisa cuatro decisiones en el código:

- **forkJoin** pide al mismo tiempo las categorías y el producto (al editar) y continúa cuando llegan ambos; así el select ya tiene sus opciones cuando se ejecuta `setValue()`.
- **opciones** es un `computed` que ofrece solo las categorías activas, más la categoría original del producto aunque esté inactiva, para no dejar el select vacío.
- **toSignal** convierte los cambios del select en un signal; con él, `categoriaInactiva` detecta si el usuario eligió una categoría inactiva y `guardar()` no envía la petición.
- **hayCategoriasActivas** muestra un aviso con un enlace a «Nueva categoría» cuando no hay ninguna activa, en lugar de un formulario imposible de completar.

Archivo: `src/app/features/productos/pages/producto-form/producto-form.ts`

```
import { Component, computed, inject, input, OnInit, signal } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { HttpErrorResponse } from '@angular/common/http';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { forkJoin } from 'rxjs';
import { erroresDeValidacion, mensajeError } from '../../../../core/utils/http-error';
import { Categoria } from '../../../categorias/models/categoria.model';
import { CategoriaService } from '../../../categorias/services/categoria-service';
import { ProductoRequest } from '../../models/producto.model';
import { ProductoService } from '../../services/producto-service';

@Component({
  selector: 'app-producto-form',
  imports: [ReactiveFormsModule, RouterLink],
  templateUrl: './producto-form.html',
  styleUrl: './producto-form.css',
})
export class ProductoForm implements OnInit {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly productoService = inject(ProductoService);
  private readonly categoriaService = inject(CategoriaService);
  private readonly router = inject(Router);

  readonly id = input<string>();

  protected readonly categorias = signal<Categoria[]>([]);
  protected readonly categoriaOriginal = signal<number | null>(null);
  protected readonly cargando = signal(true);
  protected readonly guardando = signal(false);
  protected readonly error = signal<string | null>(null);
  protected readonly erroresServidor = signal<Record<string, string>>({});

  protected readonly form = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3), Validators.maxLength(150)]],
    precio: this.fb.control<number | null>(null, [Validators.required, Validators.min(0.01)]),
    stock: this.fb.control<number | null>(0, [Validators.required, Validators.min(0),
Validators.pattern(/^\d+$/)]),
    estado: [true],
    categoriaId: this.fb.control<number | null>(null, Validators.required),
  });

  /** Opciones del select: categorías activas + la que ya tenía el producto. */
  protected readonly opciones = computed(() =>
    this.categorias().filter(c => c.estado || c.id === this.categoriaOriginal()));

  protected readonly hayCategoriasActivas = computed(() =>
    this.categorias().some(c => c.estado));

  /** Valor del select convertido en signal para validar la dependencia. */
  private readonly categoriaElegida = toSignal(this.form.controls.categoriaId.valueChanges,
    { initialValue: null });

  protected readonly categoriaInactiva = computed(() => {
    const elegida = this.categorias().find(c => c.id === this.categoriaElegida());
    return !!elegida && !elegida.estado;
  });

  protected esEdicion(): boolean {
```

```
    return !!this.id();
  }

  ngOnInit(): void {
    const id = this.id();
    if (id) {
      // Categorías y producto se piden en paralelo; se continúa cuando llegan ambos.
      forkJoin({
        categorias: this.categoriaService.listar(),
        producto: this.productoService.obtener(Number(id)),
      }).subscribe({
        next: ({ categorias, producto }) => {
          this.categorias.set(categorias);
          this.categoriaOriginal.set(producto.categoriaId);
          this.form.setValue({
            nombre: producto.nombre,
            precio: producto.precio,
            stock: producto.stock,
            estado: producto.estado,
            categoriaId: producto.categoriaId,
          });
          this.cargando.set(false);
        },
        error: (err: HttpErrorResponse) => this.fallarCarga(err),
      });
    } else {
      this.categoriaService.listar().subscribe({
        next: categorias => {
          this.categorias.set(categorias);
          this.cargando.set(false);
        },
        error: (err: HttpErrorResponse) => this.fallarCarga(err),
      });
    }
  }

  guardar(): void {
    if (this.form.invalid || this.categoriaInactiva()) {
      this.form.markAllAsTouched();
      return;
    }
    const v = this.form.getRawValue();
    const dto: ProductoRequest = {
      nombre: v.nombre.trim(),
      precio: Number(v.precio),
      stock: Number(v.stock),
      estado: v.estado,
      categoriaId: Number(v.categoriaId),
    };
    const id = this.id();
    const peticion = id
      ? this.productoService.actualizar(Number(id), dto)
      : this.productoService.crear(dto);

    this.guardando.set(true);
    peticion.subscribe({
      next: () => this.router.navigate(['/productos']),
      error: (err: HttpErrorResponse) => {
        this.guardando.set(false);
        this.error.set(mensajeError(err));
        this.erroresServidor.set(erroresDeValidacion(err));
      },
    });
  }

  private fallarCarga(err: HttpErrorResponse): void {
    this.error.set(mensajeError(err));
    this.cargando.set(false);
  }
}
```

En la plantilla, el select usa `[ngValue]` para que `categoriaId` se guarde como número. La primera opción, con valor `null` y deshabilitada, obliga a elegir una categoría.

Archivo: `src/app/features/productos/pages/producto-form/producto-form.html`

```
<section class="page">
  <h2>{{ esEdicion() ? 'Editar producto' : 'Nuevo producto' }}</h2>

  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  @if (cargando()) {
    <p>Cargando datos…</p>
  } @else if (!hayCategoriasActivas()) {
    <div class="aviso">
      No hay categorías activas. Registre o active una categoría antes de crear productos.
      <a routerLink="/categorias/nuevo" class="btn btn-primary">Nueva categoría</a>
    </div>
  } @else {
    <form [formGroup]="form" (ngSubmit)="guardar()" class="card formulario">
      <label>
        Categoría
        <select class="input" formControlName="categoriaId">
          <option [ngValue]="null" disabled>Seleccione una categoría…</option>
          @for (c of opciones(); track c.id) {
            <option [ngValue]="c.id">{{ c.nombre }}{{ c.estado ? '' : ' (inactiva)' }}</option>
          }
        </select>
      </label>
      @if (form.controls.categoriaId.touched && form.controls.categoriaId.invalid) {
        <small class="error">Seleccione la categoría del producto.</small>
      }
      @if (categoriaInactiva()) {
        <small class="error">La categoría elegida está inactiva. Elija una categoría
activa.</small>
      }
      @if (erroresServidor()['categoriaId']; as mensaje) {
        <small class="error">{{ mensaje }}</small>
      }

      <label>
        Nombre
        <input class="input" formControlName="nombre" />
      </label>
      @if (form.controls.nombre.touched && form.controls.nombre.invalid) {
        <small class="error">El nombre es obligatorio y debe tener entre 3 y 150
caracteres.</small>
      }

      <div class="fila">
        <label>
          Precio (S/)
          <input class="input" type="number" step="0.01" min="0.01" formControlName="precio" />
        </label>
        <label>
          Stock
          <input class="input" type="number" step="1" min="0" formControlName="stock" />
        </label>
      </div>
      @if (form.controls.precio.touched && form.controls.precio.invalid) {
        <small class="error">El precio es obligatorio y debe ser mayor que cero.</small>
      }
      @if (form.controls.stock.touched && form.controls.stock.invalid) {
        <small class="error">El stock es obligatorio y debe ser un entero mayor o igual a
cero.</small>
      }

      <label class="check">
        <input type="checkbox" formControlName="estado" /> Activo
      </label>

      <div class="acciones">
        <a routerLink="/productos" class="btn">Cancelar</a>
        <button type="submit" class="btn btn-primary" [disabled]="guardando()">
          {{ guardando() ? 'Guardando…' : (esEdicion() ? 'Actualizar' : 'Registrar') }}
        </button>
      </div>
    </form>
  }
</section>
```

Archivo: `src/app/features/productos/pages/producto-form/producto-form.css`

```
.formulario { display: flex; flex-direction: column; gap: 10px; max-width: 560px; }
label { display: flex; flex-direction: column; gap: 4px; font-weight: 600; }
.fila { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
.check { flex-direction: row; align-items: center; gap: 8px; }
.error { color: #b42318; }
.aviso { display: flex; align-items: center; justify-content: space-between; gap: 16px;
         padding: 16px; border-radius: 8px; background: #fff6dc; }
.acciones { display: flex; justify-content: flex-end; gap: 8px; margin-top: 8px; }
```

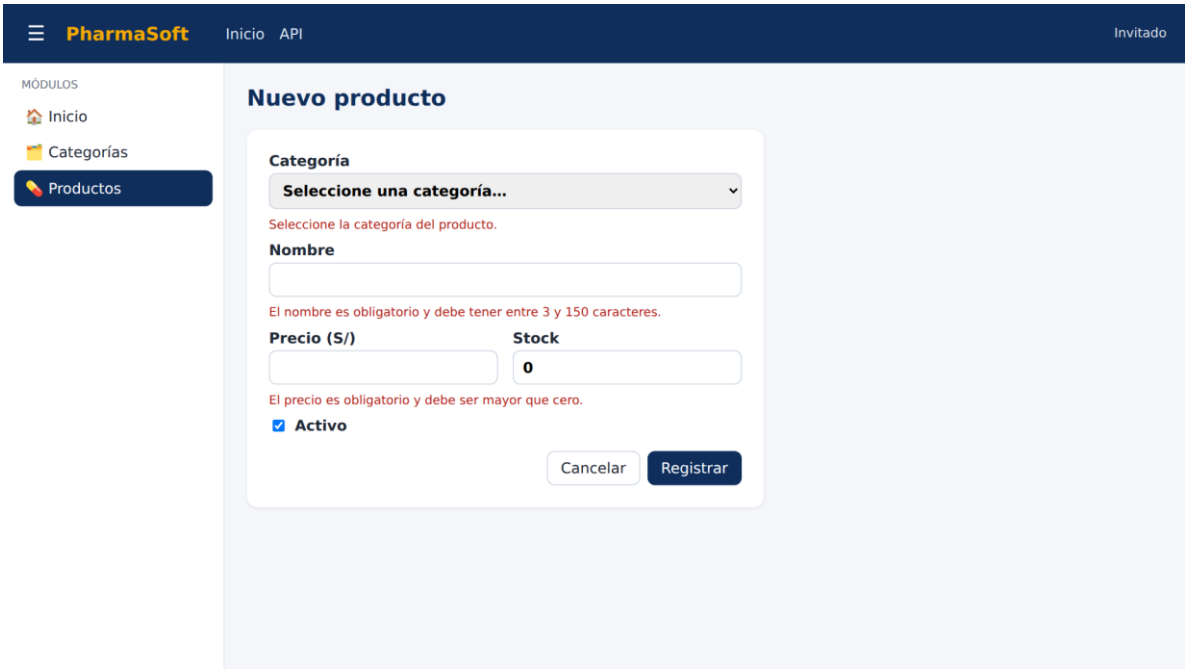


Figura 3. El formulario exige elegir una categoría antes de enviar. Nota. Elaboración propia.

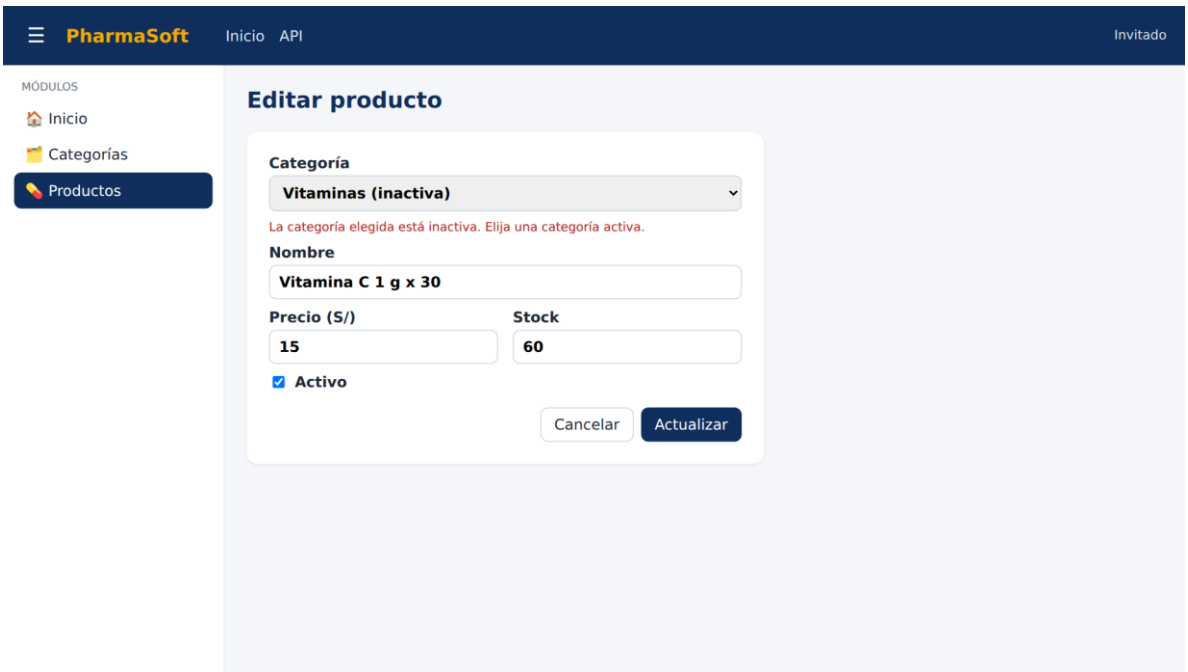


Figura 4. Edición de un producto cuya categoría está inactiva. Nota. Elaboración propia.

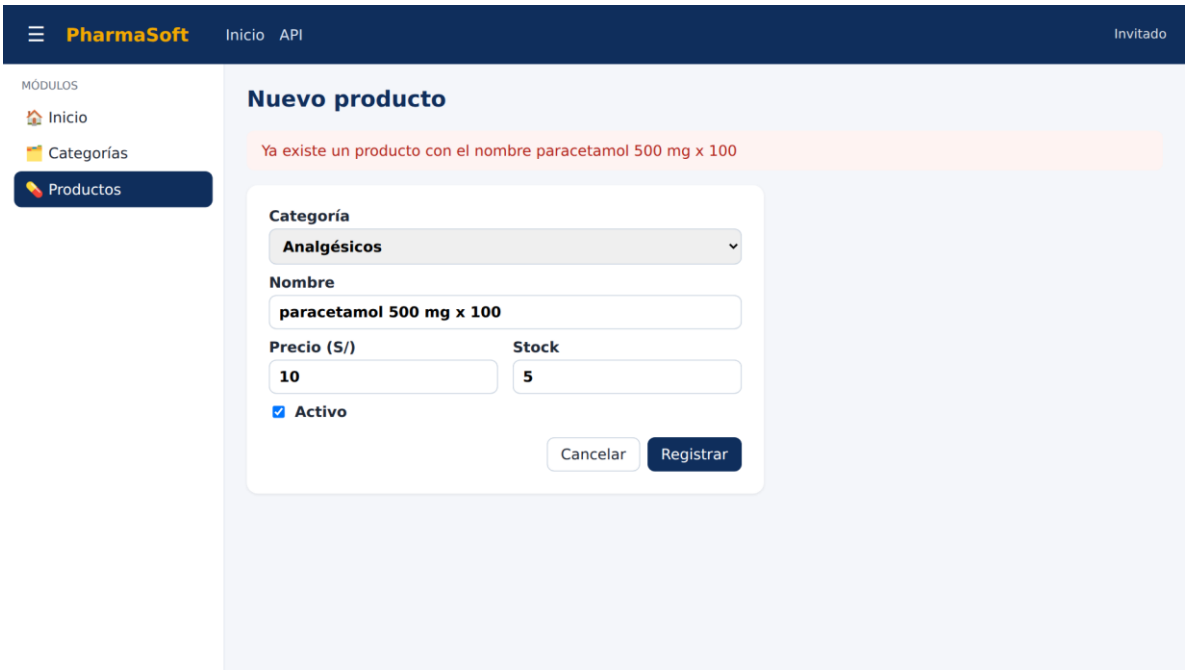


Figura 5. Nombre de producto repetido: respuesta 409 del backend. Nota. Elaboración propia.

Paso 9. Probar las dependencias

Con PharmaBackend y `ng serve` en ejecución, abre las herramientas de desarrollador (F12) en la pestaña **Red** y ejecuta estas pruebas en orden. Guarda una captura de cada una.

N.º	Acción	Resultado esperado
1	Entrar a «Productos» desde el sidebar	GET con `pagina=0&tamanio=10&ordenarPor=nombre&direccion=asc`; la tabla muestra la categoría de cada producto.
2	Hacer clic dos veces en el encabezado «Precio»	Dos peticiones: `ordenarPor=precio` con `direccion=asc` y luego `desc`.
3	Cambiar a 5 por página y pulsar «Siguiente»	Petición con `tamanio=5&pagina=1`; el paginador indica «Página 2 de …».
4	Filtrar por una categoría	La tabla muestra solo sus productos de la página actual, sin nuevas peticiones.
5	Abrir «Nuevo producto»	El select lista solo categorías activas; la inactiva no aparece.
6	Pulsar «Registrar» sin completar datos	Mensajes de validación en categoría, nombre y precio; no se envía la petición.
7	Registrar un producto válido	POST 201; al volver al listado aparece con su categoría.
8	Registrar otro producto con el mismo nombre en minúsculas	Alerta con el mensaje del backend (POST 409).
9	Editar un producto de la categoría inactiva	El select muestra «(inactiva)», aparece el aviso y «Actualizar» no envía la petición hasta elegir una categoría activa.
10	Cambiar la categoría de un producto y guardar	PUT 200; en el listado se ve la nueva categoría.

N.º	Acción	Resultado esperado
11	Dar de baja un producto	DELETE 204; tras recargar, la fila muestra «Inactivo» y su botón queda deshabilitado.
12	En «Categorías», eliminar una categoría que tiene productos	Alerta con el mensaje del backend (DELETE 409).

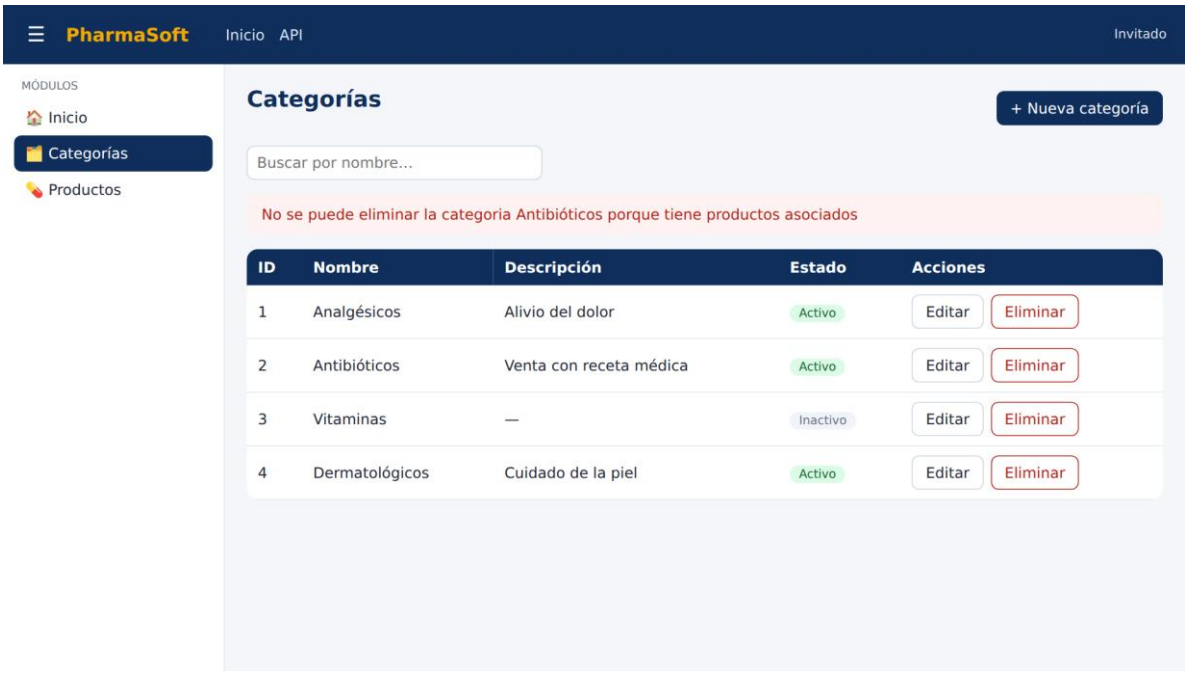


Figura 6. La categoría con productos no se puede eliminar. Nota. Elaboración propia.

Paso 10. Versionar el trabajo

```
git add .
git commit -m "feat(productos): listar productos paginados con su categoría"
git commit -am "feat(productos): ordenar, cambiar tamaño de página y filtrar por categoría"
git commit -am "feat(productos): registrar y editar con lista de categorías"
git commit -am "feat(productos): validar categoría inactiva y dar de baja"
git push -u origin feature/productos-<apellido>
```

Haz cada commit cuando esa parte funcione, no todos al final. Luego abre una solicitud de incorporación de tu rama hacia `develop`. Recuerda que `git commit -am` solo incluye archivos ya rastreados: después de crear archivos nuevos usa `git add .`.

Producto esperado

- Rama `feature/productos-<apellido>` con al menos cuatro commits que sigan la convención de mensajes y una solicitud de incorporación hacia `develop`.
- Módulo Productos en ejecución, integrado al menú, con listado paginado y ordenable, filtro por categoría, formulario con lista desplegable validada y baja lógica.
- Documento PDF `Apellido_LP2_S8_Practica.pdf` con las capturas de las doce pruebas del paso 9, mostrando la pestaña Red cuando corresponda.

Lista de cotejo

N.º	Criterio observable	Sí	No
1	El feature Productos sigue la estructura de Categorías y se carga con `loadChildren`.		
2	`PaginaResponse<T>` está en `core/models` y se usa para tipar el listado.		
3	`ProductoService` envía los parámetros de paginación con `HttpParams` y concentra las cinco operaciones.		
4	El listado muestra la categoría de cada producto con `categoriaNombre`, sin peticiones adicionales por fila.		
5	El listado permite cambiar de página, de tamaño y de orden, y filtrar por categoría.		
6	El formulario carga categorías y producto con `forkJoin` al editar.		
7	El select usa `[ngValue]`, obliga a elegir una categoría y ofrece solo las activas, más la original del producto.		
8	La SPA impide guardar un producto con una categoría inactiva y lo explica al usuario.		
9	Si no hay categorías activas, se muestra un aviso con enlace a «Nueva categoría».		
10	Los errores 400, 404 y 409 se muestran con el mensaje del backend.		
11	La baja es lógica: la fila pasa a «Inactivo» y el botón se deshabilita.		
12	La rama tiene al menos cuatro commits según la convención y una solicitud de incorporación hacia `develop`.		

3. Problemas frecuentes

Síntoma	Causa probable	Solución
El POST responde 400 en `categoriaId` aunque elegiste una categoría	El select usa `[value]` y envía el id como texto, o no se eligió ninguna opción.	Usa `[ngValue]="c.id"` y verifica en la pestaña Red que `categoriaId` sea un número.
Al editar, el select aparece en «Seleccione una categoría…»	El producto se cargó antes que las categorías, o su categoría no está entre las opciones.	Usa `forkJoin` y el `computed opciones` que incluye la categoría original.
La tabla queda vacía o muestra «undefined»	Se trató la respuesta paginada como un arreglo.	Los productos están en `resultado().contenido`.
Al ordenar aparece un error 409	Se envió un `ordenarPor` que el backend no admite.	Usa solo id, nombre, precio o stock (tipo `OrdenProducto`).
La fila no cambia después de dar de baja	No se recargó la página después del DELETE.	Llama a `cargar()` en el `next` de `darDeBaja()`.
El filtro por categoría no muestra productos que sí existen	El filtro trabaja sobre la página actual.	Es el comportamiento esperado; aumenta el tamaño de página o revisa la pregunta de la actividad autónoma.
El aviso de categoría inactiva nunca aparece	El `computed` lee `form.value` directamente, que no es un signal y no se vuelve a evaluar.	Usa `toSignal(this.form.controls.categoriaId.valueChanges, { initialValue: null })`.

4. Referencias

Angular. (s. f.-a). *Reactive forms*. https://angular.dev/guide/forms/reactive-forms

Angular. (s. f.-b). *RxJS interop with Angular signals*. https://angular.dev/ecosystem/rxjs-interop

Angular. (s. f.-c). *SelectControlValueAccessor*. https://angular.dev/api/forms/SelectControlValueAccessor

Bampakos, A., & Deeleman, P. (2023). *Learning Angular* (4.ª ed.). Packt Publishing.

RxJS. (s. f.). *forkJoin*. https://rxjs.dev/api/index/function/forkJoin